

- 进程 A 发起了一个系统调用，该系统调用在内核态运行的时间很长，例如从磁盘读取一个很大的文件，此时进程 A 处于等待状态。
- 进程 B 给进程 A 发送一个信号 (Signal)，进程 A 需要立即从内核态返回用户态处理这个信号。

在这个场景中，进程 A 被打断的系统调用应该怎么办呢？ITS 与 UNIX 做出了不一样的选择。ITS 的处理方式是：①内核保存当前 I/O 的状态（例如已经读到的磁盘位置与读取的内容）与应用上下文；②内核更新进程 A 的上下文以反映系统调用已经完成的部分工作；③从内核返回进程 A 并处理信号；④进程 A 使用更新后的应用上下文继续执行，并重新发起系统调用；⑤内核处理系统调用，从上次被中断时的位置继续执行（如从上次读取的位置继续读磁盘）。而 UNIX 的处理方式则是操作系统直接返回一个错误号，表示系统调用出错；至于“判断系统调用是否正确，如果出错该如何处理”的责任，则丢给了应用程序。

可以看到，两个操作系统的原则是不同的：ITS 是通过较为复杂的操作系统设计和实现，使系统调用的接口对于应用程序来说是简单的（接口简单，实现复杂）；而 UNIX 则避免了复杂的操作系统实现，但应用程序必须考虑调用接口可能出现的更多情况，并自己处理这些情况（实现简单，接口复杂）。

1989 年，Richard Gabriel 在一篇论文中抛出了著名的操作系统设计观点 “Worse is better”^[16]：在所有的设计考量中，简单性最重要，其中实现简单比接口简单更重要；为了简单性，必要时甚至可以牺牲部分正确性、一致性与功能完备性。他对比了两种设计方法：一种是 MIT 方法，以设计了 Common Lisp 以及 Common Lisp Object System (CLOS) 的 MIT 为代表；另一种是新泽西方法[⊖]，以设计了 UNIX 与 C 语言系统的贝尔实验室为代表。他认为，基于 Worse is better 理念的新泽西方法对于设计软件尤其是操作系统来说更为合适。

表 2.3 展现了两种设计方法在理念上的差异。MIT 方法认为：

- 设计要简单，接口简单比实现简单更重要，也就是要“将复杂留给自己，简单留给用户”。
- 在所有可观测到的方面，设计一定要正确，不正确的设计是不能容忍的。
- 设计要确保一致性，为了一致性，设计者甚至可以牺牲部分简单性与完备性。

表 2.3 MIT 方法与新泽西方法的对比

	MIT 方法	新泽西方法
简单性	接口简单比实现简单更重要	实现简单比接口简单更重要
正确性	需要 100% 正确	简单比正确更重要
一致性	必须满足一致性	可以为了简单性而牺牲一致性
完备性	必须尽可能完备	可以为了简单性而牺牲完备性

⊖ 贝尔实验室在美国新泽西州。

- 设计要尽可能地覆盖现实中各方面的重要场景，所有合理的、可预期的场景必须要覆盖到，完备性的优先级高于简单性的优先级。

而新泽西方法则认为：

- 设计要简单，但实现简单比接口简单更重要，简单原则高于其他原则。
- 在所有可观测到的方面，设计一定要正确，但为了设计简单，可以略微牺牲正确性。
- 设计要尽可能保持一致，但为了设计简单，可以牺牲一致性。例如，如果为了一致性而需要考虑一种不常见的情况，导致实现变得更复杂，那么可以不考虑这种情况。
- 设计要尽可能覆盖现实中各方面的重要场景，所有合理的、可预期的场景必须要覆盖到；但为了简单性，完备性可以让步。在保持设计简单性的前提下，可以牺牲一致性（尤其是接口一致性）来实现完备性。

现实中，很多操作系统的设计在一开始采用的是 Worse is better 的新泽西方法：先设计出覆盖基本情况、具备基本功能的操作系统来占领市场，然后根据新的需求不断迭代和完善功能，从而向 MIT 方法逼近。例如，本章提到的 Linux 0.01 版只有 8 102 行代码，而 Linux 5.7 内核则达到了 2 870 万行代码。如果在一个操作系统的起步阶段就求全责备，则要么很难坚持到实现完成，要么只能应用在某些特定的场景。本章提到的 IBM 1991 年斥资 20 亿美元设计的 Workplace 操作系统，就因为一开始设计过于复杂而最终失败。

值得一提的是，在 Richard Gabriel 的 Worse is better 的观点被广泛传播后，Nickiebn Bourbaki 写了一篇名为“Worse is better is worse”的文章^[17]对这个观点进行了批判，认为 Worse is better 的设计理念——如“牺牲接口的简单性来换取实现的简单性”——是有害的，很容易误导刚入行的新人或学生，使他们以此为理由去做些简单而偷懒的设计。有意思的是，后来人们发现，Nickiebn Bourbaki 实际上是 Richard Gabriel 的笔名。我们认为，不存在绝对正确的设计方法，要针对实际场景的需要做出合理的取舍。

2.6 ChCore：教学科研型微内核操作系统

ChCore 是一个面向教学和科研开发的微内核操作系统，其基本架构如图 2.13 所示。ChCore 采用了微内核架构的设计理念，将大部分的操作系统功能以系统服务的形式运行在用户态。以文件系统为例，在 Linux 或 Windows 中，文件系统通常作为操作系统内核的一部分直接运行在内核态；然而在 ChCore 中，文件系统不再属于内核的一部分，而是作为一个系统服务以独立

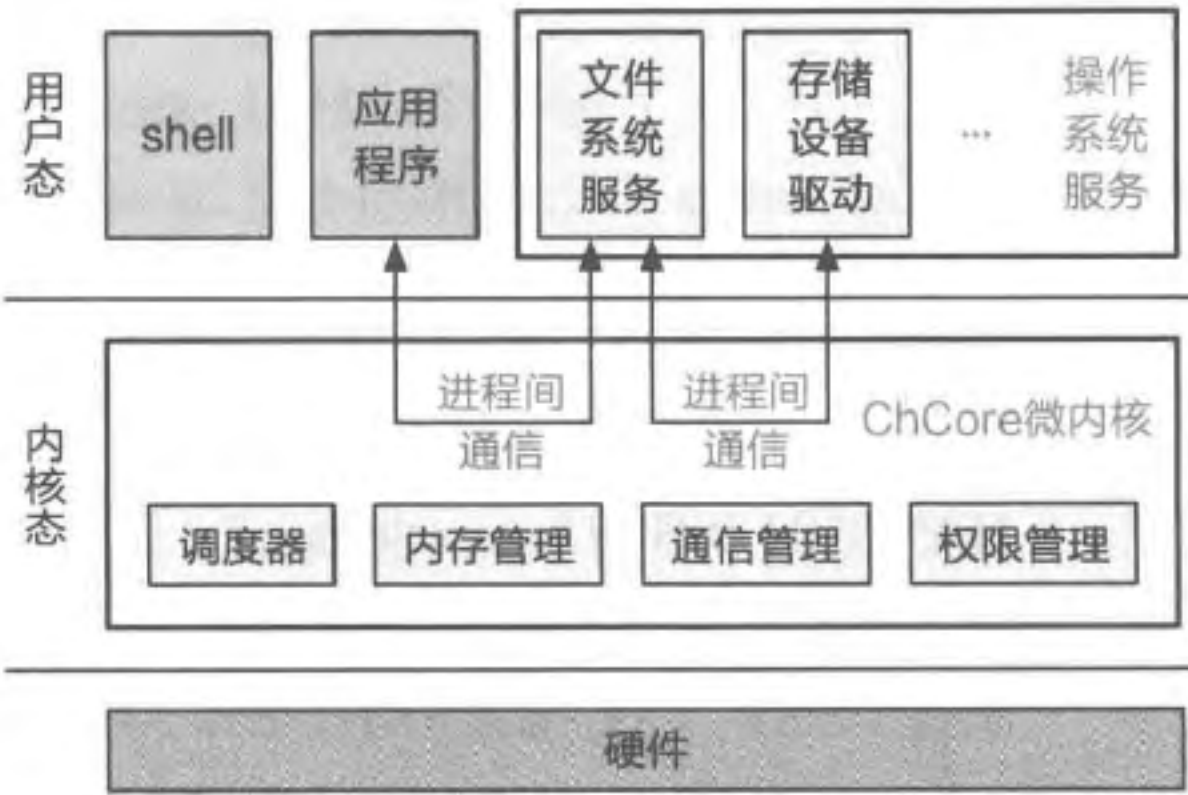


图 2.13 ChCore 系统架构示意图

进程的形式运行在用户态。当应用程序需要使用文件系统功能时，它将在 ChCore 微内核的帮助下与文件系统通信。这种设计的一个优势是显著提升了操作系统的安全性与可靠性。具体来说，文件系统的崩溃问题或安全漏洞很难影响操作系统中其他的功能组件，如内存管理、设备驱动等；同样，其他功能组件的问题也通常不会影响文件系统的运行。

在拥有特权的内核态，ChCore 仅仅运行很少的代码，负责最基础的职责与功能，主要包括内存管理、调度器、进程间通信、权限管理。值得注意的是，微内核架构只是一种操作系统设计思想，其本身并没有约定哪些功能组件应该运行在内核态。例如，一些微内核操作系统如 seL4^[10] 将内存管理也移出了内核态，从而使内核态代码量更少，同时也使对内核态代码的形式化证明变得更加方便。

总而言之，ChCore 操作系统教学版主要包括两个部分：运行在内核态的微内核和运行在用户态的若干操作系统服务。此外，ChCore 提供了一个简单的命令行终端程序（如图 2.13 中所示的 shell）作为与用户交互的界面。用户可以在 shell 中输入命令以使用系统的各种功能或运行各种应用程序。

2.7 思考题

1. 微内核与外核的主要区别是什么？它们分别有哪些优势和劣势？
2. 以下哪些是机制？哪些是策略？
 - 先进先出换页
 - $O(1)$ 调度队列
 - 基于 B+ 树的文件结构
3. 关于操作系统中的机制与策略分离，你还能想到哪些例子？
4. 一般而言，微内核架构操作系统的主要性能瓶颈在哪里？可以如何优化其性能？
5. 多内核架构有哪些优点和缺点？

参考文献

- [1] FLEISCH B D, CO M A A. Workplace microkernel and os: a case study [J]. Software: Practice and Experience, 1998, 28 (6): 569-591.
- [2] HANSEN P B. The nucleus of a multiprogramming system [J]. Communications of the ACM, 1970, 13 (4): 238-241.
- [3] SALTZER J H, KAASHOEK M F. Principles of computer system design: an